

Condizioni composte e priorità

Operatori logici, resto, priorità degli operatori

Shahram Rahatlou

Corso di Laboratorio di Calcolo, Canale Q-Z
Anno Accademico 2026-2027

SAPIENZA
UNIVERSITÀ DI ROMA

Oggi tre programmi

I primi due crescono per passi: un'idea in più alla volta

`probabilita.c`

`&&` `||` `!`

p è una probabilità valida? Condizioni composte e regola di De Morgan

`divisibilita.c`

`%` `&&` `||`

pari, multipli di 3 e 5, anno bisestile: il resto e le condizioni composte

`priorita.c`

`*` `/` `+` `-` `&&` `||`

cosa fa a*b/c? La tabella delle priorità degli operatori

Ad ogni passo: le righe nuove sono in verde acqua; il programma si compila e si esegue come sempre.

Una probabilità valida?

probabilita.c, passo 1: in matematica si scrive 0 ≤ p ≤ 1. E in C?

probabilita.c (passo 1)

```c
int main() {
    double p;
    printf("p = ");
    scanf("%lf", &p);
    if (0 <= p <= 1) {
        printf("p valida\n");
    } else {
        printf("p non valida\n");
    }
}
```

Terminale

```
$ ./app.exe
p = 0.3
p valida
$ ./app.exe
p = 5
p valida
$ ./app.exe
p = -2
p valida
```

avviso del compilatore (-Wall -Wextra)

```
result of comparison of constant 1 with boolean expression is always true [-Wtautological-constant-compare]
```

Compila, ma dice «valida» per qualsiasi p: (0 <= p) vale 0 o 1, e 0 <= 1 è sempre vero.

Due condizioni insieme: &&

probabilita.c, passo 2: p ≥ 0 e anche p ≤ 1, il AND logico

probabilita.c (passo 2)

```c
int main() {
    double p;
    printf("p = ");
    scanf("%lf", &p);

    if (p >= 0 && p <= 1) {
        printf("p valida\n");
    } else {
        printf("p non valida\n");
    }
}
```

Terminale

```
$ ./app.exe
p = 0.3
p valida
$ ./app.exe
p = 5
p non valida
$ ./app.exe
p = -2
p non valida
$ ./app.exe
p = 1
p valida
```

`0 && 0 → 0`

`0 && 1 → 0`

`1 && 0 → 0`

`1 && 1 → 1`

Fuori dall'intervallo: || e !

probabilita.c, passo 3: lo stesso controllo, scritto in tre modi

probabilita.c (finale, dall'if in poi)

```
    if (p >= 0 && p <= 1) {
        printf("p valida\n");
    } else {
        printf("p non valida\n");
    }

    int dentro = (p >= 0 && p <= 1);
    int fuori = (p < 0 || p > 1);
    printf("dentro = %d, fuori = %d, !dentro = %d\n", dentro,
fuori, !dentro);
}
```

Terminale

```
$ ./app.exe
p = 0.3
p valida
dentro = 1, fuori = 0, !dentro = 0
$ ./app.exe
p = 5
p non valida
dentro = 0, fuori = 1, !dentro = 1
```

|| (OR)

vale 1 se almeno una condizione vale 1

! (NOT)

scambia 1 e 0: !1 vale 0, !0 vale 1

Regola di De Morgan

!(A && B) = !A || !B

Negare una condizione: ! e la regola di De Morgan

Per negare un confronto composto si nega ogni parte e si scambia && con ||

`!0` → `1`

`!1` → `0`

scritto con !	equivalente	significa
`!(p >= 0 && p <= 1)`	`p < 0 || p > 1`	fuori dall'intervallo
`!(p < 0 || p > 1)`	`p >= 0 && p <= 1`	dentro l'intervallo
`!(x > a)`	`x <= a`	negazione di un confronto

Regola di De Morgan: `!(A && B) = !A || !B` e `!(A || B) = !A && !B`

Il numero è pari?

divisibilita.c, passo 1: serve il resto della divisione, l'operatore %

divisibilita.c (passo 1)

```c
int main() {
    int n;
    printf("n = ");
    scanf("%d", &n);

    if (n % 2 == 0) {
        printf("%d pari\n", n);
    } else {
        printf("%d dispari\n", n);
    }
}
```

Terminale

```
$ ./app.exe
n = 8
8 pari
$ ./app.exe
n = 7
7 dispari
```

17 % 5 = 2

il resto di 17 diviso 5: 17 = 3·5 + 2

n % 2 == 0

il resto di n diviso 2 è zero: n è pari

n % 10

l'ultima cifra di n: 2024 % 10 = 4

Multipli di 3 e di 5

divisibilita.c, passo 2: il resto con && e ||, attenzione all'ordine dei casi

divisibilita.c (passo 2, dal primo if)

```
    if (n % 2 == 0) {
        printf("%d pari\n", n);
    } else {
        printf("%d dispari\n", n);
    }

    if (n % 3 == 0 && n % 5 == 0) {
        printf("multiplo di 3 e 5\n");
    } else if (n % 3 == 0 || n % 5 == 0) {
        printf("multiplo di 3 o 5\n");
    } else {
        printf("nessun multiplo\n");
    }
}
```

Terminale

```
$ ./app.exe
n = 15
15 dispari
multiplo di 3 e 5
$ ./app.exe
n = 9
9 dispari
multiplo di 3 o 5
$ ./app.exe
n = 10
10 pari
multiplo di 3 o 5
$ ./app.exe
n = 7
7 dispari
nessun multiplo
```

L'anno è bisestile?

divisibilita.c, passo 3: divisibile per 4, ma non per 100, a meno che non lo sia per 400

divisibilita.c (finale, solo la parte nuova)

```
    if ((n % 4 == 0 && n % 100 != 0) || n % 400 == 0) {
        printf("anno %d bisestile\n", n);
    } else {
        printf("anno %d non bisestile\n", n);
    }
}
```

Terminale (ultima riga)

```
$ ./app.exe
n = 2024
anno 2024 bisestile
$ ./app.exe
n = 1900
anno 1900 non bisestile
$ ./app.exe
n = 2000
anno 2000 bisestile
```

&& prima di ||

come × prima di +: l'AND si valuta per primo

parentesi

mettile anche se non servono: gcc avvisa e chi legge ringrazia

1900 e 2000

multipli di 100: solo 2000 lo è anche di 400

Cosa fa a*b/c? La tabella delle priorità

priorita.c: chi comanda quando una riga ha più operatori

priorita.c

```
double a = 4, b = 3, c = 2;
printf("a*b/c   = %f\n", a * b / c);
printf("a/b*c   = %f\n", a / b * c);
printf("a/(b*c) = %f\n", a / (b * c));
```

Terminale

```
$ ./app.exe
a*b/c   = 6.000000
a/b*c   = 2.666667
a/(b*c) = 0.666667
```

operatori (dal più forte)	
1	! - (segno)
2	* / %
3	+ -
4	< <= > >=
5	== !=
6	&&
7	||
8	=

Esercizi

Da fare a casa, poi si controlla con un programma in C

1 Biglietto

Leggi l'età (intero). Stampa «non valida» se negativa o oltre 120; ingresso gratuito sotto i 3 anni; biglietto ridotto sotto i 14 anni o da 65 in su; intero negli altri casi.

prova con:

```
2     12     40     80     -4
```

2 Divisibile

Leggi un intero n. Stampa «divisibile» se n lo è per 4 ma non per 6, altrimenti «no».

prova con:

```
8     12     20     14
```

3 Media pesata

Leggi voto e CFU di due esami e calcola la media pesata con i CFU. Attenzione alle parentesi.

$$\text{media} = \frac{\text{voto}_1 \cdot \text{cfu}_1 + \text{voto}_2 \cdot \text{cfu}_2}{\text{cfu}_1 + \text{cfu}_2}$$

prova con:

```
voto1 cfu1 voto2 cfu2
24 6 30 9
18 12 30 6
27 9 27 9
```

Compila con `gcc -Wall -Wextra -o app.exe nome.c -lm` (-lm serve per le funzioni di math.h)

Riepilogo

1	**&& || !**	AND, OR, NOT: una condizione composta; per negare, De Morgan
2	**%**	resto della divisione intera: pari, multipli, anni bisestili
3	**Priorità**	! prima di * / %, poi + -, poi i confronti, poi && e infine ||: nel dubbio, parentesi